Building a Modern DXP

Plan and architectural specification — v2

An Elixir/Phoenix/Ash-based DXP with a unified component model, a framework-agnostic Vite plugin family for authoring, and progressive-disclosure authoring UX over a Matrix-inspired asset graph.

Status	Draft for internal discussion
Date	4 May 2026
Version	v2 — supersedes v1 of 3 May 2026
Audience	Solo / small founding team
Origin	Distilled from architecture conversation, May 2026

0. What changed in v2

This version supersedes the 3 May draft. It folds in feedback from the annotated v1 review and a follow-up architecture discussion. The platform thesis is unchanged; several specific decisions and framings have been corrected or sharpened.

Area	Change
Authoring framework	v1 named Astro as *the* recommended path. v2 reframes around a Vite plugin family that transforms SSR output from any compatible framework (Astro, Vue, others) to a single Phoenix-targeted artefact format. No framework is privileged at the contract level.
Component model	Unified. Pages, layouts, and components are no longer separate types — they are all components with role metadata. Any component can declare an expected layout. This is a meaningful simplification of the Squiz 'design / paint layout / page / component' type family.
Component runtime loading	v1 implied components compile into the Elixir release. v2 specifies that component artefacts (HEEx templates, JS bundles, CSS, manifest) are deployed as files to object storage and loaded at render time. Sites subscribe to component versions independently of the platform release cycle.
Hydration model	Clarified. The server emits complete, SEO-perfect HTML — not placeholders. Props are emitted alongside as the rendering input. Hydration boots the client component with the same props and attaches behaviour to the existing DOM rather than re-rendering.
Search	Reversed. v1 specified OpenSearch (JVM). v2 starts with Postgres full-text search, with Meilisearch (Rust, single binary) as the upgrade path. OpenSearch only if a customer specifically needs it. No JVM in the standard architecture.
Framework	Ash is now the resource framework. Most of section 4 (asset model) and section 8 (mutation API) become Ash resource declarations with policies, paper trail, state machines, and Oban integration as Ash extensions.
Editor views	Both ship in Phase 1, not Phase 1+2 split. Content-shaped editor is the default; asset-map/developer mode is a toggle available immediately. AshAdmin provides a credible power-user view essentially for free.
Component contract surface	Clarified. The contract describes the manifest (props, slots, events, modes, role) and the artefact paths. It does not constrain how components are styled or implemented internally — those are the developer's domain.

1. Executive summary

This document specifies a Digital Experience Platform (DXP) designed as a credible alternative to Squiz DXP and to the broader incumbent set (Adobe AEM, Sitecore, Acquia, Optimizely). It is informed by direct experience working inside Squiz, by analysis of Squiz Matrix's open source ancestor (MySource Matrix), and by a review of contemporary headless and composable platforms.

The architectural thesis is in three parts. First, the asset-graph data model that Squiz Matrix inherited from MySource Matrix is genuinely ahead of modern headless CMSes for governance and reuse, and should be retained. Second, the way Matrix exposes that model to editors — as raw assets in a tree — is the largest single source of editor friction, and should be replaced with a progressive-disclosure UX that hides assets behind content-shaped affordances by default while keeping the asset-map view available from day one. Third, Matrix's custom XML templating and PHP runtime should be replaced with an Elixir/Phoenix/Ash substrate plus a unified component model where pages, layouts, and components are all components with role metadata, deployed as files to the platform and loaded at render time.

> **Position in one sentence**
>
> Same governance benefits as Squiz Matrix, half the clicks, modern stack, and a unified component model that supports any SSR-capable authoring framework via a single Vite plugin family.

Key decisions taken in this document

- Elixir/Phoenix as the platform substrate, with Ash as the resource framework and PostgreSQL as the primary datastore.
- Matrix-style asset DAG as the data model, with a Read/Write/Admin permission model inherited down the tree, implemented via Ash policies plus a permission cache.
- Progressive disclosure of the asset model in the authoring UI — content-shaped editor by default, asset map / developer mode as a toggle available from Phase 1.
- Unified component model: pages, layouts, and components are one type, distinguished by role metadata. Any component can declare an expected layout.
- A stable, versioned component contract as the primary public interface. The contract describes the manifest and artefact paths; it does not constrain component internals.
- A Vite plugin family transforms SSR output from Astro, Vue, and other compatible frameworks to the Phoenix component artefact format. No authoring framework is privileged at the contract level.
- Component artefacts (HEEx templates, JS bundles, CSS, manifest) deploy as files to object storage. Sites subscribe to component versions and updates do not require a platform redeploy.
- Server emits complete, SEO-perfect HTML alongside the props that produced it. Hydration boots the client component with the same props and attaches behaviour to the existing DOM.
- Four runtime modes per component instance: static SSR (cached), LiveView, Phoenix Channels with client hydration, external-driven with client hydration.
- Search starts with Postgres full-text search via AshPostgres. Meilisearch as a sidecar from Phase 3 if needed. OpenSearch only on specific customer demand.
- Vue 3 SPA against a Phoenix API for the authoring UI; LiveView reserved for simpler admin screens; AshAdmin as a free-tier power-user view in Phase 1.

2. Goals and non-goals

2.1 In scope

- Multi-tenant SaaS DXP capable of hosting many sites per tenant with shared design system and component library.
- Authoring UI that a non-technical editor can use without ever seeing the asset tree, plus a developer-mode toggle that exposes the asset graph for power users.
- Developer experience that allows component authoring in any framework supported by the Vite plugin family (Astro and Vue at launch), without knowing Elixir.
- Governance baseline matching Squiz Matrix: per-asset permissions, workflow, audit log, version history, safe-edit preview.
- Search good enough to compete with Funnelback for the 80% case, starting with Postgres full-text and with a clear path to Meilisearch and RAG-based conversational search.
- Hosting model: SaaS first, with a self-host option for customers who require it.

2.2 Out of scope (initial release)

- E-commerce engine. Integrate with Shopify, Stripe, or BigCommerce rather than building one.
- Marketing automation (email campaigns, drip sequences). Integrate with existing tools.
- A native mobile authoring app. Web-first; mobile authoring deferred until web UX is settled.
- Customer Data Platform (CDP) with full identity stitching. Integrate RudderStack or similar for v1.
- Translation memory and computer-assisted translation. Hooks for it; not a built-in.

2.3 Explicit non-goals

- Feature parity with Adobe Experience Manager. AEM costs orders of magnitude more for a reason; matching it is the wrong target.
- Replacing Funnelback's 25-year ranking engine. Postgres FTS plus Meilisearch covers the 80% case without dragging a JVM into the architecture.
- Privileging any single authoring framework at the contract level. The Vite plugin family is the universal adapter; framework choice is the developer's.
- Becoming the customer's full martech stack. The DXP is the experience layer; CRM, CDP, and email live elsewhere with good integrations.

3. Architectural principles

These principles constrain every design decision below. They are listed in priority order; where two principles conflict, the higher-listed one wins.

Principle	What it means
Progressive disclosure of complexity	The asset model is a backend concern. Editors see content, not assets. The asset map / developer mode is reachable via a single toggle from day one — it is never the default but it is never deferred either.
One source, many runtimes	Components are authored once and compile to multiple runtime targets (Phoenix server-render, client hydration bundle, optionally LiveView). The component contract is the stable public interface.
Unified component model	Pages, layouts, and components are one type with a role metadata field. The contract is identical; the role determines composition rules. This collapses Squiz's 'design / paint layout / page / component' type family.
Implicit creation of implied assets	If creating asset A necessarily implies the existence of asset B, the system creates B with sensible defaults and surfaces it only when the editor wants to deviate.
Server-rendered, SEO-complete by default	The server emits complete final HTML readable by search engines. Hydration is opt-in per instance and attaches behaviour to existing DOM rather than re-rendering.
Standards at the contract surface, freedom inside	W3C DTCG for design tokens, JSON Schema for component props, OpenAPI for the content API, WCAG 2.2 for accessibility. The contract uses these standards. How components are styled and implemented internally is the developer's domain.
Asset graph as single source of truth	All writes go through one Ash mutation API. The asset map and the content-shaped editor are both projections of the same graph; neither bypasses the other.
Multi-tenant from day one	Single-tenant assumptions are technical debt the moment the second customer signs. Postgres row-level security via Ash multitenancy from the first commit.
Single-runtime preferred	Adding a JVM (or any heavyweight separate runtime) next to an Elixir app is a step backwards operationally. Search, queues, caching, and observability all favour Elixir-native or single-binary sidecars.
Bounded surface area	Build the seven non-negotiable capabilities exceptionally well. Integrate everything else. Feature accumulation is not a strategy.

4. The asset model

The data model is a directed acyclic graph of typed assets. Every page, image, user, group, component instance, redirect, form submission store, metadata schema and workflow is an asset. This is identical in spirit to MySource Matrix's model, modernised onto Postgres and expressed as Ash resources.

4.1 Resources, not tables

Where v1 sketched core tables, v2 expresses them as Ash resources. The underlying Postgres shape is similar; the declaration is denser and the surrounding capabilities (actions, policies, multitenancy, paper trail, state machine, archival) are wired in by extensions.

```
defmodule Core.Assets.Asset do
  use Ash.Resource,
    domain: Core.Assets,
    data_layer: AshPostgres.DataLayer,
    extensions: [
      AshPaperTrail.Resource,    # versioning
      AshStateMachine,           # status machine
      AshArchival.Resource,      # soft-delete (the trash)
      AshOban                    # background jobs hook
    ]

  postgres do
    table "assets"
    repo Core.Repo
  end

  multitenancy do
    strategy :attribute
    attribute :tenant_id
  end

  attributes do
    uuid_primary_key :id
    attribute :type, :atom, allow_nil?: false
    attribute :role, :atom           # for components: :page, :layout, :component
    timestamps()
  end

  state_machine do
    initial_states [:draft]
    default_initial_state :draft
    transitions do
      transition :submit_for_review, from: :draft,     to: :review
      transition :approve,           from: :review,    to: :live
      transition :start_safe_edit,   from: :live,      to: :safe_edit
      transition :commit_safe_edit,  from: :safe_edit, to: :live
      transition :archive,           from: [:live, :draft], to: :archived
    end
  end
end
```

With versioning, relationships, and policies wired in via further blocks on the same resource:

```
    paper_trail do
      change_tracking_mode :full_diff
    end

    relationships do
      has_many :outgoing_links, Core.Assets.AssetLink, destination_attribute: :parent_id
      has_many :incoming_links, Core.Assets.AssetLink, destination_attribute: :child_id
      has_many :permissions,    Core.Assets.Permission
    end

    actions do
      defaults [:read]
      create :create do ... end
      update :update do ... end
      destroy :archive, primary?: true
    end

    policies do
      policy action_type(:read) do
        authorize_if Core.Policies.HasAssetPermission, level: :read
      end
      policy action_type([:create, :update]) do
        authorize_if Core.Policies.HasAssetPermission, level: :write
      end
      policy action_type(:destroy) do
        authorize_if Core.Policies.HasAssetPermission, level: :admin
      end
    end
  end
```

Companion resources follow the same pattern: AssetLink for the DAG edges, Permission for the principal-level grants, MetadataSchema and MetadataValue for typed metadata, Workflow and WorkflowRun for approval flows. Versioning is handled by AshPaperTrail rather than a hand-rolled asset_versions table.

4.2 The implication graph

Each asset type declares which other assets it implies. The system creates these implicitly with sensible defaults. The implication graph is a small Spark DSL extension on top of Ash resources that drives editor UI generation, asset bootstrapping, and cascade behaviour on move/delete.

```
defmodule Core.Content.Page do
  use Ash.Resource,
    extensions: [Core.Implications]

  implications do
    implies :url do
      default      &Core.URL.compute_from_slug/1
      surfaced_as  :inline_field   # editor sees a URL field, not an asset
      on_delete    :convert_to_redirect
    end

    implies :metadata_record do
      default      :empty_for_schema
      surfaced_as  :advanced_panel
      on_delete    :cascade
    end
  end
end
```

4.3 The dual-view discipline

The same asset graph is exposed through two UIs from Phase 1: the content-shaped editor (default for editors) and the asset map / developer mode (default for system administrators and developers, available to anyone via toggle). Both UIs share the same Ash mutation API; neither has shortcuts that bypass the asset layer. A right-click affordance switches between views: "Open as asset" from the editor, "Show in editor" from the asset map.

CHANGED IN V2 — Both views from Phase 1

v1 implied the asset map view was a later phase. v2 ships both views from Phase 1. AshAdmin provides a credible asset-map view essentially for free, which is enough to satisfy the developer-mode requirement until the bespoke Vue 3 view is built.

Why this matters

Squiz Matrix exposes the asset model directly to editors. The cumbersomeness of "create a URL asset, then create a page, then link them" is the single biggest source of editor friction in Matrix and the most-cited weakness in reviews. Keeping the data model and replacing the affordance layer is the largest single competitive lever available against Squiz.

5. The unified component model

The component model is the most novel part of this architecture. v2 unifies pages, layouts, and components into a single type distinguished by role metadata. It separates three concerns that are usually bundled: the authoring format, the component contract, and the runtime mode.

5.1 Pages, layouts, and components are one thing

In Squiz, designs, paint layouts, pages, and components are different asset types with different rules and different override behaviour. Editors have to learn which type does what. v2 collapses this: a component is something that takes props and slots and produces HTML. A page is a component with role :page. A layout is a component with role :layout. The contract is identical; the role only changes composition rules.

A component can declare multiple roles. A component can declare an expected layout. Layouts themselves can have layouts. Composition is recursive slot-filling, the same way React, Vue, and Astro converged on internally.

> **CHANGED IN V2 — Unified type, simpler surface**
> v1 still implicitly carried Squiz's separation of pages, layouts, and components. v2 collapses them into one type. The page asset still exists (because it has implied URL, metadata, etc.) but it references a component by role rather than being its own kind of view.

5.2 The component contract

Every component, regardless of authoring format or role, ships with a manifest matching this contract. The contract is a versioned spec, published openly, and adapter authors target it. The contract describes the manifest and the artefact paths — it does not constrain how components are styled, written, or implemented internally.

Field	Purpose
name	Globally unique within the component set.
version	Semver. Sites pin major versions; minor/patch flow through automatically.
roles	Which roles this component fills: :page, :layout, :component. Multiple permitted.
expects_layout	Optional. If set, declares this component should be wrapped by a layout matching the given role/contract.
props	JSON Schema. Drives editor UI, runtime validation, and IDE autocomplete.
slots	Named slots with declared accept-types. Used for composition. No scoped slots in v1.
events	Named events with payload schemas. Wired differently per runtime mode.
modes	Which runtime modes the component supports: static, live_view, channels, external.
a11y	Declared accessibility commitments (semantic role, keyboard interactions, ARIA contract).

Field	Purpose
artefacts	Paths to the build outputs: render_server (HEEx), render_client (JS bundle), styles (CSS bundle).

> **CHANGED IN V2 — Styles are an artefact, not a contract field**
>
> v1 listed styles as a peer of props and slots. That implied the manifest declared styles, which it doesn't. In v2, styles are part of the artefacts section — a path to the CSS bundle the build emitted. How the component author writes those styles is their concern.

5.3 Authoring through a Vite plugin family

v2 reframes authoring around a single Vite plugin family that transforms SSR output from any compatible framework to the Phoenix component artefact format. There are two authoring paths, both first-class:

- **Phoenix-native.** Components written directly as Phoenix function components / HEEx / LiveView. No compilation step. First-class for Elixir developers.
- **Vite-plugin-transformed.** Components written in Astro, Vue, or any other framework with an adapter in the plugin family. The Vite plugin pre-renders the component, extracts the manifest, bundles the client-side JS and CSS, and emits the Phoenix-targeted artefacts.

Astro and Vue ship with first-party adapters from launch. Other frameworks (Svelte, Lit, etc.) can be added by writing additional adapters against the same plugin contract. The platform team maintains the plugin core and the launch-day adapters; community contributions are welcome but not guaranteed to track upstream framework version bumps.

5.4 The four runtime modes

A component declares which modes it supports; an instance on a page picks one mode. The same component can run in different modes on different pages — a Card is static on the homepage (cached at the edge) and LiveView on a logged-in dashboard (real-time updates).

Mode	Server cost	Client cost	Update mechanism	Best for
Static SSR	Render once, cache	None (until hydrated)	Page reload only	Marketing pages, articles, listings
LiveView	Stateful process per session	~30 KB shim	Phoenix WS, server controls DOM	Dashboards, complex forms
Channels + client	Pub/sub, no per-session state	Component runtime	Phoenix WS, client controls DOM	Live widgets with local state
External + client	None per session	Component runtime	AJAX, third-party WS, GraphQL subs	Components talking to other systems

5.5 SSR-complete HTML and the hydration model

> **CHANGED IN V2 — SEO-complete server render, props as input**
>
> v1 described hydration as if the server emitted markers and the client filled them in. That was wrong and it would have been bad for SEO. v2 specifies that the server emits complete final HTML — the page is fully readable by search engines and accessible without JS. The props that produced the HTML are emitted alongside as the input the client needs to reach the same state.

The server-rendered HTML is the truth. It is complete, semantically correct, and indexable. Hydration is the act of attaching client-side behaviour to that existing DOM — not re-rendering it. The flow:

1. Server renders the component to complete HTML using the resolved props and slots. The HTML is what users and search engines see, and it is what the cache stores.
2. The same props are emitted alongside the HTML as a JSON payload (in a script type="application/json" block adjacent to the component or as data attributes — whichever is cleaner for the component shape).
3. If the component requires hydration (any mode other than pure static), a small client-side runtime locates the component in the DOM, locates its props, and lazy-loads the matching client bundle.
4. The client component boots with the same props that produced the server HTML. By construction it produces the same DOM. Hydration is therefore an attach-behaviour step, not a re-render.
5. From that point the client component owns its part of the DOM and can update in response to user interaction or external events.

This eliminates the entire class of hydration-mismatch bugs that plague React-style hydration, because the client doesn't independently render — it reuses the server's DOM and only attaches listeners and reactivity to it.

```
// The client-side hydration spine — conceptually ~150 lines
const registry = new Map();

export function register(name, loader) {
  registry.set(name, loader);
}

async function hydrateAll() {
  const markers = document.querySelectorAll('[data-component]');
  for (const el of markers) {
    const name = el.dataset.component;
    const loader = registry.get(name);
    if (!loader) continue;

    // The props that produced this server-rendered DOM
    const propsScript = document.querySelector(
      `script[data-props-for="${el.dataset.componentId}"]`
    );
    const props = propsScript ? JSON.parse(propsScript.textContent) : {};

    // Boot the client component with those same props.
    // It attaches behaviour to the existing DOM rather than re-rendering.
    const Component = await loader();
    Component.hydrate(el, props);
  }
}
```

Production polish on top of this spine: IntersectionObserver-based lazy loading for below-the-fold components, requestIdleCallback-based loading for non-critical components, shared-runtime deduplication so multiple components using the same framework runtime ship one copy. Roughly 1500–3000 lines of JavaScript total for a polished v1.

5.6 Layout composition and cycle prevention

Any component can declare an expected layout via expects_layout in its manifest. The render pipeline composes them: render the layout, render the wrapped component into the layout's default slot, recurse for nested components. This is uniform whether the component is a page being wrapped by a site shell, a form field being wrapped by a field-group layout, or a comment being wrapped by a comment frame.

Because layouts are components and components can have layouts, cycles are structurally possible. Cycle prevention is two-layered:

- **Static analysis at component publish time.** When a component is uploaded, the platform validates that its layout chain does not transitively reference itself. Cycles fail the upload with a clear error.
- **Render-time depth limit.** The renderer maintains a stack of components currently being rendered; if a component being rendered is already on the stack, the render aborts with a structured error. Catches anything that escaped static analysis.

5.7 Component subscription and version pinning

Sites subscribe to components by name and version range, not to specific component instances. This is closer to npm semver than to Squiz's per-asset component references.

- **Indirect by default.** A page asset references a component by name (article-page). The site declares a subscription with a version range (article-page: ^2.0.0). Updates within the major version flow through automatically.
- **Pinnable per asset.** An individual page can override the site default and pin a specific version (article-page@2.1.3) when stability matters more than freshness — for example, a campaign page during a launch window.
- **Subscription is itself an Ash resource** with paper trail. Version changes are auditable; rollback is one update away.

6. Component runtime loading

This is the section v1 lacked. It addresses how components get from a developer's Git repo to running on the platform without a platform redeploy. The model maps closely to what Squiz Component Service does, achieved with fewer moving parts.

6.1 The component pipeline

1. Component author works in a Git repo with their Astro / Vue / HEEx source.
2. The Vite plugin family pre-renders the component, extracts the manifest, bundles the client JS and CSS, and emits a directory of artefacts: render_server.heex, render_client.js, styles.css, manifest.yaml.
3. A sync mechanism uploads the artefact directory to the platform. Two paths: a CLI (dxp deploy components/) for developer iteration, and a Git webhook for production CI/CD. Both produce the same outcome.
4. The platform validates the manifest, runs cycle-prevention static analysis on layout chains, stores the artefacts in object storage under tenants/{tenant_id}/components/{name}/{version}/..., and writes a ComponentVersion record to the database.
5. Sites subscribed to this component via a matching version range are notified. A domain event fires, an Oban worker invalidates affected cache entries.
6. On the next request to a page using the component, Phoenix renders the page using the new component version. The rendered HTML is written to the cache.

6.2 The artefact format

Component artefacts are stored as files in object storage, not as compiled BEAM modules. Specifically:

- render_server.heex — a HEEx template string. Loaded into Phoenix at render time via Phoenix.LiveView.HTMLEngine. Compiled-to-AST once on first use, cached in an ETS table keyed by {name, version}.
- render_client.js — a JavaScript bundle for client-side hydration. Served via the platform's CDN. Lazy-loaded by the hydration runtime when the component requires hydration.
- styles.css — a scoped CSS bundle. Served via the platform's CDN. Linked from the rendered page as a <link> tag.
- manifest.yaml — the component contract. Parsed once and cached as a struct. Drives prop validation, editor UI generation, layout composition, and subscription matching.

Storing HEEx as a string rather than as a precompiled BEAM module avoids the operational complexity of dynamic :code.load_binary/3 and BEAM compilation in the build pipeline. The performance cost is moderate (a few milliseconds for first compile, sub-microsecond for subsequent renders from the ETS cache) and is bounded by the render-and-cache pipeline — production renders are cache hits.

6.3 The render-and-cache pipeline

Most renders most of the time should be cache hits. The cache is the system's primary performance mechanism.

- **Cache key:** {asset_id, page_component_version, layout_component_version, child_component_versions, locale, audience_segment}. Different combinations produce different cached HTML.
- **Cache storage:** tier 1 in process memory (Cachex), tier 2 in Redis or similar for larger horizons. CDN sits in front of the application cache for the truly hot pages.
- **Invalidation:** Phoenix PubSub events from Ash actions trigger Oban workers that mark affected cache entries stale. Granular — updating an asset only invalidates pages that reference it.
- **Regeneration:** on cache miss, Phoenix re-renders using current component versions and writes the result back. This is incremental static regeneration at the platform level.
- **LiveView opts out.** LiveView components are rendered live per session and do not participate in the cache. This is by design — LiveView is the mode for genuinely dynamic, per-user experiences.

6.4 Storage shape

What	Where	Why
Component artefacts (HEEx, JS, CSS)	Object storage (S3 / R2 / MinIO)	Heavy, immutable per version, served via CDN.
Component metadata (versions, manifests, subscriptions)	Postgres via Ash resources	Structured, queryable, relational.
Compiled HEEx ASTs	ETS in process memory	Hot path, microsecond access, regenerable from object storage.
Rendered HTML cache	Cachex tier 1, Redis tier 2	Hot path, larger working set than ETS handles.
Asset graph (pages, links, permissions)	Postgres via Ash resources	Transactional, relational, audit-logged via AshPaperTrail.
Image binaries	Object storage	Heavy, served via imgproxy + CDN.
Image metadata	Postgres via Ash resources	Structured, alongside the asset graph.

7. Reference architecture

Sized for a small founding team building a multi-tenant SaaS DXP. Component choices are biased toward open source, self-hostable, single-runtime-where-possible, and operationally simple.

Layer	Choice	Notes
Edge	Cloudflare	CDN, WAF, Workers for edge personalization.
Web/render tier	Phoenix + Ash + HEEx	Server-rendered by default. LiveView for interactive surfaces.
Resource framework	Ash + AshPostgres	Resources, actions, policies, multitenancy. Replaces hand-rolled Ecto contexts.
Authoring UI	Vue 3 SPA + Pinia	Talks to Phoenix API. Asset map / developer mode toggle from Phase 1.
Power-user view (Phase 1)	AshAdmin	Free admin UI over Ash resources. Serves as developer mode until the bespoke Vue view exists.
Content API	AshJsonApi + AshGraphql	REST and GraphQL generated from Ash resources. OpenAPI emitted automatically.
Versioning	AshPaperTrail	Replaces hand-rolled asset_versions table. Full diff tracking.
State machine	AshStateMachine	Asset status transitions (draft → review → live → safe_edit → archived).
Soft delete	AshArchival	The trash. Preserves rows, marks deleted, supports restore.
Identity	Keycloak + AshAuthentication	Keycloak for SSO/SAML. AshAuthentication for service-to-service tokens.
Component runtime	HEEx via Phoenix.LiveView.HTMLEngine	Compiled from any framework via the Vite plugin family. See section 6.
Search (Phase 1–2)	Postgres FTS via AshPostgres	tsvector + pg_trgm + unaccent. No new runtime.
Search (Phase 3+)	Meilisearch sidecar (optional)	Single Rust binary. No JVM. Provider-agnostic Phoenix indexing layer.
DAM	S3-compatible + imgproxy sidecar	R2 or MinIO. Metadata as Ash resources alongside content.
iPaaS	n8n (self-hosted)	Closest open source equivalent to Squiz Connect.
CDP	RudderStack (open source)	Defer building your own. Integrate, not invent.
Analytics	PostHog + Plausible	Product analytics + privacy-friendly site analytics.
Background work	Oban via AshOban	Indexing, image processing, cache invalidation, webhooks, scheduled publishes.
Cache	Cachex (tier 1) + Redis (tier 2)	Render cache, ETS for hot AST cache.

Layer	Choice	Notes
Primary store	PostgreSQL 16+	Single DB, AshPostgres multitenancy via row-level security.
Observability	OpenTelemetry → Grafana stack	Tempo for traces, Loki for logs, Prometheus for metrics. Sentry for app errors.

7.1 Multi-tenancy strategy

Single Postgres database. AshPostgres multitenancy with the :attribute strategy enforces tenant_id scoping on every query at the framework level — there is no path to cross-tenant data access through the Ash API. Per-tenant Postgres schemas are reserved for tenant-private extensions (custom asset types, custom workflow handlers).

> **CHANGED IN V2 — Ash multitenancy, not hand-rolled RLS**
>
> v1 specified Postgres row-level security as the multitenancy mechanism. v2 uses Ash's :attribute multitenancy strategy, which enforces tenant scoping at the framework level on every query. RLS remains as a defence-in-depth layer at the database level, but the primary enforcement is in Ash.

7.2 Search strategy and the case against OpenSearch

v1 specified OpenSearch. v2 reverses that decision. OpenSearch runs on the JVM, which adds a second runtime to a project whose appeal is single-runtime simplicity. For the scales this product targets in Phase 1–3 — universities with 50,000 pages, councils with a few hundred thousand documents — Postgres full-text search is not just adequate but typically faster than OpenSearch (no JVM warmup, no GC pauses, no network hop).

The phased approach:

- **Phase 1–2:** Postgres FTS via AshPostgres. tsvector calculations declared on resources, pg_trgm for fuzzy matching, unaccent for accent-insensitive search, optionally pgvector for semantic search. Free, no new runtime, sub-50ms latency typical.
- **Phase 3+:** Meilisearch as a sidecar if Postgres FTS hits a wall (relevance tuning, scale, complex faceting). Meilisearch is a single Rust binary, no JVM, REST-driven. The Phoenix indexing layer is provider-agnostic; swapping providers is a configuration change.
- **OpenSearch only on demand.** If a specific customer has a Funnelback-grade requirement, OpenSearch becomes a deployment option for that customer. It is not in the standard stack.

7.3 The hot paths

- **Public page render.** Cloudflare edge cache → Phoenix render cache (Cachex/Redis) → on miss, Phoenix renders from HEEx in ETS → Postgres for content via Ash. Target: P95 < 200 ms at origin on cache miss, P95 < 50 ms at edge on hit.
- **Authoring save.** Vue SPA → Phoenix API → Ash mutation in transaction → AshOban job to reindex and invalidate cache. Target: P95 < 300 ms for the round trip the editor sees.

- **Image upload.** Direct-to-S3 from browser via signed URL → Oban derivative job → imgproxy on-the-fly resize at delivery. Target: original visible to editor immediately, derivatives ready within 5 seconds.
- **Search query.** Phoenix → Postgres FTS via Ash → response with highlighting and facets. Target: P95 < 100 ms.
- **Component publish.** CLI upload → manifest validation → cycle analysis → object storage write → ComponentVersion record → AshOban cache invalidation. Target: deploy-to-live < 30 seconds.

8. Roadmap

Sized for a single engineer at first, growing to a small team. Each quarter ends with a deployable, internally-usable product. The riskiest assumption (the Vite plugin family compiling to HEEx strings) is validated in Phase 0 before any product investment.

> **CHANGED IN V2 — Phase 1 timeline tightens with Ash**
>
> v1's roadmap had Phase 1 at roughly 12 weeks. With Ash absorbing much of what v1 specified as hand-written code (resources, actions, policies, multitenancy, paper trail, state machine, archival, admin), Phase 1 becomes 6–8 weeks for a focused engineer. Phase 2 and onward shift earlier proportionally.

8.1 Phase 0 — validation (1 weekend)

Single goal: prove the Vite plugin can compile a non-HEEx component to a HEEx string that Phoenix can render.

- Take three existing Astro components from the current Squiz workplace pipeline.
- Write a minimal Vite plugin that emits HEEx template strings from the Astro components.
- Render them in a hello-world Phoenix app via Phoenix.LiveView.HTMLEngine. Verify props, slots, and scoped CSS round-trip cleanly.
- Smoke-test that the SSR HTML is SEO-complete and that hydration boots a client-side counterpart.
- If the compiler is under 1000 lines of Node and the output feels clean, proceed. If not, reconsider before sinking product time into the architecture.

8.2 Phase 1 — minimum viable platform (~6–8 weeks)

- Phoenix + Ash skeleton with PostgreSQL. AshPostgres multitenancy from the first migration.
- Ash resources for Asset, AssetLink, Permission, MetadataSchema. AshPaperTrail for versioning. AshStateMachine for status. AshArchival for soft-delete.
- AshJsonApi REST endpoints. AshAuthentication for tokens. Keycloak for SSO.
- Vue 3 authoring UI shell. Login. Asset tree view. Single content-shaped editor for the 'page' asset type.
- AshAdmin enabled as the developer-mode view from day one. The bespoke Vue asset map can come in Phase 2.
- Vite plugin (Astro adapter) in production. Three components from the existing Squiz pipeline render via Phoenix.
- Render-and-cache pipeline v1: Cachex tier-1, Postgres-event-driven invalidation via AshOban.
- Static SSR rendering for one site type. Component subscription as an Ash resource.
- DAM v1: direct-to-S3 upload, imgproxy derivatives.
- Audit log on every Ash mutation (free via AshPaperTrail).
- Deploy to a single tenant for dogfooding. Migrate one real site to it.

8.3 Phase 2 — DXP-shaped (~3 months)

- Per-asset Read/Write/Admin permissions via Ash policies, inherited down the asset DAG, with an ETS-backed permission cache.
- Workflow + approvals as Ash resources. Safe-edit / preview-as-published.
- Multi-site under one tenant. Per-site design system and component library.
- Postgres FTS via AshPostgres expressions. Faceted search. Search analytics.
- Forms engine v1 — form is just a component with role :form; submissions are an Ash resource.
- Localization — per-locale content variants on Ash resources, fallback chains, locale-aware URLs.
- AshGraphql alongside AshJsonApi.
- Vite plugin Vue adapter alongside Astro adapter.
- Bespoke Vue asset map view alongside AshAdmin.

8.4 Phase 3 — competitive (~3 months)

- Component runtime modes 2–4: LiveView, Phoenix Channels with client hydration, external-driven.
- n8n integration for iPaaS-shaped workflows. Webhooks. Connector library.
- RudderStack ingestion. Audience segments. Rules-based personalization.
- A/B testing framework with edge personalization on Cloudflare Workers.
- Conversational AI search via RAG over the Postgres FTS index (and pgvector).
- Meilisearch sidecar as an optional upgrade for customers who need it.
- Public component contract specification published.

8.5 Phase 4 — differentiation (Q4 and Y2 H1)

- Agentic authoring: file-to-DXP (PDF/Figma to draft page using existing components), conversational page building.
- Content Intelligence crawler — accessibility, SEO, broken links, AI-discoverability across the customer's whole estate.
- ISO 27001 certification work (started in Q1; formalised here).
- Public component registry. Plugin SDK. Vite plugin adapters for Svelte, Lit, etc.
- Self-host distribution for customers who require it.

9. Initial specifications

These are the small, concrete specifications that would let an implementation begin. They are deliberately incomplete — the goal is to capture the shape, not to lock down every detail.

9.1 The component manifest

Every compiled component ships with a manifest. This is the contract artefact.

```
# component.manifest.yaml — emitted by every Vite plugin adapter
name: article-page
version: 2.1.0
authoring_format: astro

roles: [page]                      # this component fills the :page role

expects_layout:                    # optional — declares a wrapper
  matches_role: layout
  default: standard-layout

props:
  $schema: https://json-schema.org/draft/2020-12/schema
  type: object
  properties:
    title:         { type: string, maxLength: 200 }
    body:          { type: string }
    hero_image_id: { type: string, format: uuid }
    is_favourited: { type: boolean, default: false }
  required: [title]

slots:
  default:
    accepts: html
    hydration: preserve

events:
  toggle_favourite:
    payload:
      type: object
      properties:
        id: { type: string, format: uuid }
      required: [id]

modes: [static, live_view, channels, external]

a11y:
  role: article
  keyboard:
    - { key: Enter, on: focused-toggle, action: toggle_favourite }

artefacts:
  render_server: ./render_server.heex
  render_client: ./render_client.js
  styles:        ./styles.css
```

9.2 The asset mutation API

All writes from any UI go through Ash actions. AshJsonApi generates the REST endpoints from the resource definitions; the same actions back the GraphQL API via AshGraphql.

```
POST /api/v1/assets
  Create an asset. Body: { type, parent_id, link_type, body }
  Returns the created asset and any implied assets created alongside it.

PATCH /api/v1/assets/:id
  Update an asset. Body: { body, status, metadata }
  AshPaperTrail creates a new version automatically.
  Implied assets cascade as declared in the implication graph.

POST /api/v1/assets/:id/links
  Add a secondary or notice link to the asset DAG.

POST /api/v1/assets/:id/permissions
  Grant or revoke. Body: { principal_id, level }

POST /api/v1/assets/:id/workflow/transitions
  Move through workflow steps. Body: { transition, comment }

DELETE /api/v1/assets/:id
  Soft-delete via AshArchival. Implied assets cascade as declared.

All Ash actions emit domain events to a Phoenix PubSub topic, which AshOban
workers subscribe to for indexing, derivatives, cache invalidation, webhooks.
```

9.3 Repository layout

```
/
|-- apps/
|   |-- core/                       # Phoenix umbrella app, Ash domain
|   |   |-- lib/core/              # Ash resources — assets, components, workflows
|   |   |-- lib/core_web/          # HTTP endpoints, LiveView admin screens
|   |   `-- priv/repo/migrations/
|   |
|   |-- authoring/                  # Vue 3 SPA — the editor UI
|   |   |-- src/views/
|   |   |-- src/components/
|   |   `-- vite.config.ts
|   |
|   `-- component-runtime/          # client-side hydration runtime
|       `-- src/
|
|-- packages/
|   |-- component-contract/         # the manifest spec, JSON Schemas, types
|   |-- vite-plugin-core/           # plugin family core, adapter contract
|   |-- vite-plugin-astro/          # Astro adapter
|   |-- vite-plugin-vue/            # Vue adapter
|   |-- adapter-astro-squiz/        # existing Squiz target — kept for migration
|   |-- design-tokens/              # W3C DTCG tokens, shared across targets
|   `-- components-core/            # the starter component set
|
|-- infrastructure/
|   |-- docker-compose.yml          # local dev — postgres, keycloak, minio
|   |-- terraform/                  # production infra
|   `-- k8s/                        # helm charts for self-host customers
|
`-- docs/
    |-- component-contract.md       # the published spec
    |-- architecture/
    `-- runbooks/
```

9.4 The first ten pull requests

In order. Each is small, mergeable, and produces something visible.

1. Phoenix umbrella app + Ash + AshPostgres skeleton. Hello-world endpoint. AshAdmin mounted at /admin.
2. Asset Ash resource with AshPostgres + AshPaperTrail + AshStateMachine + AshArchival + multitenancy. CRUD via AshAdmin works end-to-end.
3. AssetLink Ash resource. DAG traversal helpers (ancestors, descendants, paths) as Ash calculations.
4. Permission Ash resource + Ash policy module that resolves Read/Write/Admin via inheritance. ETS-backed permission cache.
5. AshJsonApi endpoint generation. POST /api/v1/assets round-trip with versioning verified.
6. Vite plugin core + Astro adapter. Three components from the existing pipeline compile to HEEx strings.
7. Component Ash resource. Component artefacts upload to MinIO via a CLI tool. ComponentVersion records tracked.

8. Phoenix render path: load HEEx from object storage, compile to ETS-cached AST, render with props from Ash. First end-to-end page render from authored content.
9. Vue 3 SPA shell. Login via Keycloak. Asset tree view (read-only). Content-shaped editor for the 'page' asset type, saving via the AshJsonApi mutation.
10. Render-and-cache pipeline: Cachex tier-1, AshOban-driven invalidation on Ash domain events. Static SSR for a 'page' asset using the Astro-compiled component set. Visible site.

After PR 10, the system is the smallest end-to-end DXP that renders real pages from authored content using the full component pipeline, with Ash policies enforcing per-asset permissions and AshAdmin available as the developer-mode view.

10. Risks and mitigations

Risk	Mitigation
Vite plugin family is harder than expected	Phase 0 (1 weekend) validates this before any product investment. The existing Astro-to-Squiz pipeline already proves the architectural pattern works; the Phoenix target is incremental work.
Ash framework lock-in	Ash is opinionated. If you fight it, you'll be miserable. Mitigation: Ash 3.x is mature with commercial backing through Alembic; the lock-in risk is real but manageable, and the productivity gain for a solo founder dramatically outweighs the cost.
Elixir talent pool is small	Solo founder phase makes this irrelevant. By the time hiring starts, the codebase plus Ash plus the published contract are the documentation. Senior Node or Ruby developers ramp in 4–8 weeks.
Multi-tenant isolation gets harder under load	AshPostgres multitenancy plus Postgres RLS as defence-in-depth is well-trodden territory at the scales this product will hit for years. Database-per-tenant is the escape hatch; design the data layer to allow it (no cross-tenant joins, ever).
Postgres FTS hits a wall for a customer	Meilisearch as a sidecar upgrade. Single Rust binary, no JVM. The Phoenix indexing layer is already provider-agnostic. Switching is a configuration change, not a rewrite.
Component runtime loading complexity	HEEx-as-string + ETS AST cache is the simplest credible model. Avoids dynamic BEAM compilation entirely. Per-render performance bounded by the cache hit rate, not the renderer's speed.
Layout cycles in components	Two-layered prevention: static analysis at publish time, render-time depth limit as backstop. Squiz hit this and it is well-understood territory.
CDP is the second moat	Don't build one. RudderStack open source handles event collection and warehouse activation. Identity stitching is a v2/v3 concern at best.
ISO 27001 timing	Plan compliance work from Q1 even though certification lands in Y2. Audit log (free via AshPaperTrail), RBAC (free via Ash policies), encryption at rest, secrets management — all need to be there from the first PR.
Scope creep into marketing automation, e-commerce	Hard "out of scope" stance from section 2.2. Every feature request gets evaluated against "is this an experience-layer concern?" If not, integrate.
Squiz catches up	Possible but slow. They have 25 years of installed-base inertia and trained editors who expect the asset map. The Notion-style affordance layer plus the unified component model are wedges precisely because they cannot ship them without breaking existing customers.

11. Open questions

Genuinely undecided items. Each needs a deliberate choice before the relevant phase starts; leaving them implicit produces drift.

- **Authoring UI framework — Vue 3 or LiveView?** Working assumption is Vue 3 SPA, given familiarity and SPA suitability for a complex authoring surface. AshAdmin covers the developer-mode case in Phase 1, which buys time for the decision. Confirm before Phase 1 ships the editor view.
- **Custom-element interop for client-rendered components.** Web Components let consumers embed components in non-Phoenix sites cleanly. They also impose Shadow DOM costs. Decision: ship as Web Components only at the consumption boundary, not internally. Confirm before Phase 3.
- **Pricing model.** Per-tenant flat rate, per-site, per-active-editor, per-rendered-page? No data yet. Decide before second customer.
- **Self-host distribution model.** Open core with proprietary cloud features? Fully open source with paid hosting? Source-available with a commercial licence? Each has different trade-offs. Decide before Phase 4.
- **Component contract versioning policy.** Semver per component is clear. Versioning the contract *itself* across breaking changes needs a written policy. Defer until first breaking change is needed.
- **Migration tooling for existing Squiz Matrix sites.** Real differentiator if it works, large project if it doesn't. Initial position: provide a content export bridge, not a full automated migration. Revisit once first Squiz customer asks.
- **Scoped slots in v2 of the contract.** v1 of the contract excludes scoped slots (where a slot's content can receive data from the parent component). This is a deliberate restriction to keep the contract simple and portable. Revisit when a real component needs the capability.
- **Naming.** Working name needed for the project. The platform's name shapes its positioning more than is comfortable to admit.

Appendix A — vocabulary

Term	Meaning in this document
Asset	Any addressable typed object in the system: page, image, user, URL, redirect, component instance, workflow schema, etc.
Asset DAG	The directed acyclic graph of asset_links. One asset can have many parents via primary, secondary and notice links.
Component	A unit that takes props and slots and produces HTML. Pages, layouts, and 'small' components are all components, distinguished by role metadata.
Role	The composition role a component fills: :page, :layout, or :component. A component can declare multiple roles.
Component contract	The published, versioned specification every compiled component must satisfy. Constrains the manifest and artefact paths, not component internals.
Component subscription	A site's declaration that it uses a component name with a version range. Updates within the range flow automatically.
Vite plugin family	The set of Vite plugins (one core, one per supported framework) that transform SSR-capable component sources to the Phoenix component artefact format.
Runtime mode	How a component instance is rendered and updated on a given page: static, LiveView, channels, or external.
Implication graph	A Spark DSL extension on Ash resources that declares which other assets a given asset type implies, with defaults and cascade behaviour.
Dual view	The discipline that the same asset graph is exposed through two UIs (content-shaped editor and asset map / developer mode), both projections of the same data.
Progressive disclosure	The principle that the asset model is a backend concern hidden by default; power-user controls are reachable but not in the editor's face.
Ash	An Elixir resource framework. Provides resources, actions, policies, multitenancy, paper trail, state machines, archival, admin, JSON:API and GraphQL generation.
HEEx	Phoenix's HTML-aware EEx templating. The component artefact format the Vite plugin family targets.
DTCG	W3C Design Tokens Community Group spec. The format used for design tokens shared across all targets.
RLS	Row-level security in Postgres. Defence-in-depth layer beneath Ash multitenancy.

Appendix B — what this document is not

- It is not a business plan. It says nothing about pricing, sales motion, target customer segment in detail, or funding.
- It is not a complete specification. Many small decisions are deferred. The goal is the shape, not every detail.

- It is not a commitment. Every decision here is reversible in Phase 0. The point is to make decisions consciously, not by drift.
- It is not an attack on Squiz. Squiz Matrix is a thoughtful piece of software with 25 years of real-world use. Where the document is critical, it is critical of choices that made sense in 2003 and don't in 2026.

Appendix C — what changed from v1, in detail

For readers comparing this to the 3 May draft, the substantive changes were:

1. Authoring framework reframed: from "Astro recommended, others community" to a Vite plugin family supporting Astro and Vue as first-class adapters with no privileged framework.
2. Component model unified: pages, layouts, and components collapsed into one type with role metadata, eliminating Squiz's design / paint layout / page / component type family.
3. Component runtime loading specified: artefacts deploy as files to object storage, HEEx loads via Phoenix.LiveView.HTMLEngine, ETS caches compiled ASTs, sites subscribe to versions independently of the platform release cycle.
4. Hydration model corrected: server emits complete SEO-perfect HTML; props are emitted as the rendering input; hydration attaches behaviour to existing DOM rather than re-rendering.
5. Search reversed: from OpenSearch (JVM) to Postgres FTS via AshPostgres in Phase 1–2, with Meilisearch as the upgrade path from Phase 3. OpenSearch only on customer demand.
6. Ash adopted: most of section 4 (asset model) and section 8 (mutation API) become Ash resource declarations with policies, paper trail, state machines, archival, Oban integration, and admin as Ash extensions.
7. Editor views: both content-shaped editor and asset map / developer mode ship in Phase 1, not split across phases. AshAdmin provides developer mode for free in Phase 1.
8. Component contract clarified: the contract describes the manifest and artefact paths only. Component internals — styling, implementation, framework-specific features — are the developer's domain.
9. Phase 1 timeline tightens to 6–8 weeks because Ash absorbs much of what was hand-written code in the v1 plan.